

ForgeGIS Studio™

Technical Brief

VERSION 1.2.6 · OCTOBER 2026

Forged For Speed. Refined by Craft.

SEAGLASS FOUNDRY LLC

The browser workspace where the Forge suite is seen and run — a live map, a pipeline canvas, a batch runner, and an agent that acts on the map you are looking at.

Executive Summary

ForgeGIS Studio™ is where the Forge suite becomes visible. The engines do the work — ForgeGIS™ computes on the GPU, ForgeData™ finds and serves data, ForgeMind™ turns plain language into validated pipelines — but they speak only libraries and machine protocols. Studio is the browser workspace that puts a live map, a pipeline canvas, a batch runner and a conversational agent in front of a person, so the suite can be seen, exercised and evaluated.

Studio is not a general-purpose GIS desktop, and it does not try to be one. It is an operator and demonstration surface: a place to show that a GPU-resident pipeline runs interactively, that raster and vector operations compose into branching graphs, that an agent can act on a live map, and that the whole stack runs on one host with nothing exposed to the network. It is deliberately thin over the engines, so it can be re-cut to a customer's task without re-plumbing the compute underneath.

1 host	**3** engines	**R · V · M**
the whole suite on one workstation — Studio bound to loopback, reached only from that machine's browser	GPU compute, the data catalog and the agent behind one workspace	raster, vector and mixed pipelines — linear chains run on the canvas, branching graphs in batch

Within the suite Studio is an optional component, chosen on the Seaglass Control Center™'s Components screen, installed and started by it, and served on port 8081. It is one of two map surfaces — Seaglass Globe™ is the other — and the rule between them is simple: **the window you type into is the map that answers.**

New in 1.2.6

Release 1.2.6 is a validation and hardening release: one new capability, and a long list of answers made right.

- **Batches on the compute broker**: raster batches go to the suite's compute broker when the Control Center has installed it, and fall back to the local runner for vector and mixed batches, inputs off the broker's shared storage, or a broker that does not answer.
- **Elevations in their own units**: every elevation readout prints the vertical units the data states — feet stay feet — and says "unknown units" when nothing does, instead of assuming metres.
- **Measurements on the WGS84 ellipsoid**: Measure lengths and areas and the exported scale bar follow the engine onto the ellipsoid, checked against GeographicLib and pyproj values.
- **Rasters placed truthfully**: a raster with no georeferencing is marked "position unknown" instead of drawn near Null Island, and a rotated GeoTIFF the reader cannot place is refused with its remedy; the viewshed palette and legend read the engine's 0 / 255 encoding.
- **Hardening**: a DNS-rebinding guard on Studio's local server, Tools-panel calls validated exactly as MCP validates them, and versioned contracts on the agent bridge and on the engines Studio links.

Recently shipped

- **1.2.3** — the digital-twin panel: run, pause, resume, checkpoint, stop and fork what-ifs, a recording browser and a replay strip, with twin bodies authored on the canvas.
- **1.2.4** — graduated vector symbology, attribute-table export to GeoJSON, GeoPackage or CSV, batch dry runs with a durable history and a termination reason on every run, and a round of security hardening.
- **1.2.5** — Globe-parity map chrome, KML, KMZ and zipped-shapefile drops, XYZ, WMS and WMTS layers by URL, per-cell raster reads, chat that answers on Studio's own map, and QGIS import warnings.

What ForgeGIS Studio Is

The suite is engine-first by design. ForgeGIS is a library and an MCP server; ForgeData a catalog and an MCP server; ForgeMind a compiler and control plane. None of them draws a map. Studio is the component that makes the suite tangible: a browser workspace, backed by a local server, that loads data, runs operations, shows results on a live map and hosts the agent conversation.

Studio's job is demonstration and operation, not parity with a desktop GIS. What it sets out to prove is what makes the Forge suite different — GPU-resident pipelines that run interactively, raster and vector graphs that branch and join, an agent acting on a real map, single-host deployability — and to be re-shaped quickly when a customer needs those capabilities in the exact shape of their workflow. It is not sold separately.

Architecture

Studio is a thin, local, three-tier application.

- **Frontend** — a React single-page application using MapLibre GL for the map and ReactFlow for the pipeline graph. It talks to the local server over HTTP and receives live status — backend device, VRAM, heap, system RAM, job progress — over a WebSocket.
- **Server** — a Javalin (Jetty) HTTP and WebSocket server on Java 17, bound to IPv4 loopback. It holds the session state, the operation executor pool and the handles for the rasters and vectors that pipelines operate on. Its API covers operations and pipeline apply, raster and vector transport, the catalog, batch jobs, filesystem roots, tools, the twin, geocoding, roads, elevation, data sources and the agent.
- **Engines** — ForgeGIS operations run **in-process** — raster operations on the server's GPU backend, vector operations on parallel CPU — through the shared operation catalog, which unions the raster, vector and cross-type registries and stamps each operation with its kind and input ports. ForgeData is **embedded as a Java library**, giving Studio catalog listing, area-windowed extraction and presence verification over the same federated catalogs the rest of the suite reads. ForgeMind is reached over HTTP, and acts on Studio's map through the `studio-mcp` bridge.

Three properties are deliberately true. **Loopback by default**: nothing on the LAN can reach Studio unless an operator rebinds it, and then it warns on every boot. **One process for compute**: there is no second process to marshal a raster across for an interactive apply. **One session, many surfaces**: the canvas, the batch runner and the agent work over the same handles, and the agent draws on the map the operator is looking at — not a copy.

The Map and Your Data

Map chrome (1.2.5). Right-click the map for *copy coordinates*, *centre* and *zoom in here*, *measure from here*, *bookmark here*, *drop a point entity*, *what's here* and *load DEM here*. A navigation strip carries zoom, a north-up compass and a Home button that frames the visible layers. The same gestures work on the Globe, so moving between the two feels like one product.

Your own files (1.2.5). KML, KMZ and shapefiles load straight into the session. From the browser a shapefile must be **zipped** together with its `.shx`, `.dbf` and `.prj` — a bare `.shp` is refused, because its sidecars have to travel with it — while a shapefile beside its sidecars also loads from the server's filesystem and from a batch folder. Zip and KMZ uploads are vetted before extraction — entry count, declared size, exactly one `.shp`, duplicate or Windows-illegal member names refused by name — and capped on real bytes as they inflate. Vector drops are never auto-published to the catalog.

Tile services by URL (1.2.5). Add your own XYZ, WMS or WMTS layer by pasting its address, and the browser remembers it across restarts. The service must be in Web Mercator and allow cross-origin requests, and there is no capabilities discovery.

Large Tools-panel results (1.2.5). ForgeGIS stops inlining geometry above 10 000 vertices on seven of its geometry tools. When a Tools-panel call hits that cap, Studio supplies an output path in a sandboxed spool, pruned after six hours, and draws the result from the file when it is EPSG:4326; otherwise it shows a named notice instead of a wrong drawing. A large result from a projected source is refused, and a geometry result with no CRS at all is a named notice rather than a guess.

Rasters. Catalog-backed loads, drops and pipeline outputs render on the map. The value under the pointer and the Measure panel's point analysis now read through the ranged-read primitive ForgeGIS added with `sample_point` — one cell, all bands, or a 3 × 3 window — instead of downloading the whole raster on every call, and NoData is judged per band, at the float precision the map paints with.

Placed, or flagged (1.2.6). A raster with no georeferencing is marked *position unknown* and is not drawn, rather than placed near Null Island with its pixels read as degrees; a windowed catalog load says how far it was downsampled. A rotated GeoTIFF the reader cannot put on an axis-aligned grid is refused with its remedy (`gdalwarp`) instead of loading as its bounding box with stretched cells. A catalog window with no valid cell says *No data in this window* instead of loading an empty layer.

Elevation in its own units (1.2.6). Elevation readouts — the right-click header, point analysis, the profile and the selected DEM's own values — carry the vertical units, datum and source ForgeData reports, and print "m" only for metres, "ft" or "US survey ft" for feet, and "(unknown units)" when nothing says. The Cut/Fill design level names the DEM's units too.

Measurements (1.2.6). Measure lengths and areas and the exported scale bar are computed on the WGS84 ellipsoid, following ForgeGIS, and pinned against GeographicLib and pyproj values; they differ from earlier spherical figures. The panel leads with the answer and lists a path's legs.

Palettes that follow the op. A result's palette follows the parameters that produced it, so slope in radians is not painted on the degrees ramp. The viewshed palette and legend read ForgeGIS's encoding — 255 visible, 0 hidden or out of range, as `gdal_viewshed` writes it — with the classes labelled in words.

Vector display. Every map-facing vector surface is served in EPSG:4326 with its CRS stated; a CRS that cannot be reprojected is refused with a named reason, and downloads stay in the native CRS. Graduated symbology (quantile, equal-interval, natural breaks) comes with a counted legend, and the attribute table filters by expression and exports GeoJSON, GeoPackage or CSV. A layer cut at a feature limit says so — "500+ features (cut)" — rather than looking complete.

The Pipeline Canvas

Operations from the ForgeGIS catalog appear as nodes; connecting their handles builds a pipeline; the topology can branch and merge rather than run as a single line.

Type-valid by construction. Raster, vector and mixed raster-to-vector pipelines are authored on one canvas. The palette gates operations by the selected node's kind — vector operations off a vector node, a raster-to-vector operation off a raster node — so a Polygonize → Buffer chain that crosses types is expressed directly and the graph is always valid. Operation categories come from the server, and class names are shown as readable labels.

Where each kind of graph runs. Linear raster, vector and mixed chains cook on the canvas as you build them. Branch, merge and sub-model nodes, and graphs with several inputs, are authored on the canvas and **run in the Batch Runner**. Project files, recipes and share URLs carry linear pipelines; branching graphs round-trip through the pipeline library.

One saved format. Pipelines serialise to a versioned format recording the pipeline type, each operation's kind, inputs and dependencies — enough to reconstruct a branching graph faithfully. The pipeline library and the batch runner share one format. After a hard reload a node's applied parameters are not restored, so replaying that node is refused rather than guessed.

QGIS models, with warnings (1.2.5). Pipelines converted from QGIS models carry their import warnings as a badge in the library and a banner in the Batch Runner — warn, don't block — including the one case that runs but answers differently (a renamed parameter whose meaning changed, such as the Z factor).

The Batch Runner

The batch runner executes saved pipelines — raster, vector and mixed — across a folder of inputs, writing one output per input in the pipeline's terminal type: GeoPackage by default for vector terminals, GeoJSON where an operator wants it.

- **On the compute broker (1.2.6).** When the Control Center has installed the suite's compute broker, raster batches are submitted to it and spread across its GPU workers; a submit whose outcome is unknown is retried as the same job so it cannot run twice, and the history and report come back from the broker. Vector and mixed batches, inputs under no shared storage root, and a fresh request to a broker that does not answer run on the local runner instead.
- **Per-tile resource pre-flight.** On the local runner, as each tile is dispatched, Studio estimates its peak VRAM through the resource-estimation contract ForgeGIS exposes and checks it against the available budget; a tile that would not fit is recorded as failed rather than discovered by crashing. Interactive applies use the same estimator; batch keeps a quarter of available VRAM in reserve, while an interactive apply may use it all.
- **Dry run.** A dry run lists what would run and where outputs would land before you commit; it does not check VRAM, raster readability or parameter fit.
- **Built to run unattended.** Batch work runs at batch priority so it cannot starve interactive operations; each operation reports progress; a tile outside the pipeline's anchor is skipped with a reason rather than failing the batch; every run ends with a termination reason and a durable report — with the estimated VRAM per tile — that survives a restart, and a run cut off by a restart is recorded as interrupted.
- **One at a time, locally.** The local runner takes one batch at a time; on the broker, concurrency is the broker's job.

Capacity, honestly. The status bar reports the backend device, its VRAM budget and use, JVM heap and system RAM, shifting from green to amber to red as the budget tightens.

The Agent in Studio

The most distinctive thing Studio does is let a natural-language agent act on the operator's own map.

It acts on this map. Ask in Studio's chat and ForgeMind can act on the canvas — place markers and named entities, draw overlays, move the camera, push reports — through the **`studio-mcp` bridge**, an authenticated channel that exposes Studio's map as MCP tools. Every turn to a ForgeMind that supports per-request surfaces — the suite's does — carries `surface: "studio"`, so an answer lands on Studio's map even when the Globe is also running, and the scene travels with the question as structured context: entities sent as a point or bounding box rather than their vertices, so a large entity never overflows the request. A ForgeMind with no Studio surface wired refuses the turn with a named reason rather than answering blind. In the suite, the Control Center wires the bridge and mints the agent's key whenever Studio is installed. An agent call into a page that has been closed is refused rather than acted on unseen.

A versioned contract (1.2.6). The bridge publishes its surface as studio-mcp contract 1.0.0: 33 listed tools with their full input schemas and a structured error shape, gated in the build so a change cannot ship unversioned. Dataset listings the agent asks for are capped and say when they were truncated.

Authenticated, not open. The bridge is reached through an agent-key exchange, not a shared token pasted into a dialog. The browser uses a lazily minted session that is safe only under loopback; the bridge uses a separate, token-validated path that will not re-mint on a stale session. An agent key binds to the most recently connected tab.

Results that persist. Named entities the agent places are recorded as durable scene entities in the ForgeData catalog — the catalog row is the source of truth, the session a cache over it. After a restart they are adopted back with their identities intact, so "the entity I placed last turn" still resolves. Transient overlays stay transient by design.

Data Access

Studio reads through ForgeData rather than owning file I/O, which keeps one catalog and one extraction strategy across the suite.

- **Catalog-backed loads** — an operator browses registered datasets, including the machine's shared catalog, and loads one; Studio never reimplements the catalog's knowledge of where data lives or how to reach remote tiers.
- **Area-windowed reads** — given an area of interest, Studio asks for just that window, sized under Studio's raster cap before it reaches the GPU — a canyon's worth of pixels, not a continental tile — and remote sources Studio could never open directly become readable.
- **Presence verification** — a registered-but-missing dataset is caught at pick time with a clear message, not at read time with an opaque file-not-found.
- **Pinned contracts (1.2.6)** — Studio pins the MCP contracts of the engines it links, ForgeData's contract 2.0.0 among them; at boot it compares them with the snapshots the engines ship; a ForgeGIS operation or tool whose contract moved incompatibly is switched off with a refusal that names both versions, and ForgeData drift is logged.

Security Posture

Studio's HTTP surface is single-user, single-host by design, and that model is defended by default — loopback bind, guards and caps — not left to assumption.

- **Loopback by default.** The server binds to 127.0.0.1. If an operator rebinds outward, Studio logs a warning on every boot while it is off loopback; the prerequisites for any hosted deployment — real authentication, CSRF protection, rate limiting, TLS, an audit log and a re-review of the sandbox — are named, not implied.
- **A mediated path perimeter.** Every route that accepts a caller-controlled path runs it through a sandbox. Raster uploads are confined to an upload directory with a write-extension allowlist and no overwrite, and vector drops are

staged under a server-minted name and deleted after reading; catalog reads are checked against the resolved file path, treating the catalog row as the dangerous input; opted-in filesystem roots are canonicalised, resolved through symlinks, checked against a deny list, and scoped to the session that added them; tool calls and the agent's catalog loads are checked against an operator-only sandbox that cannot see roots a browser session added.

- **Bounded resources.** Upload and pixel caps, a pre-dispatch VRAM estimate gate, a bounded executor pool that refuses when full, and a per-operation timeout bound the worst case; handles idle beyond a timeout are evicted unless pinned, and vector layers live within a heap budget.
- **Hardened sessions (1.2.4).** Sessions expire; cross-site writes and non-loopback WebSocket origins are refused; the session cookie is HttpOnly and SameSite.
- **DNS rebinding refused (1.2.6).** Every request and WebSocket upgrade must name a loopback `Host` with Studio's port, or it is refused — so a web page that re-points its own domain at 127.0.0.1 cannot reach Studio, the agent bridge's routes included — and a write from another local page (`same-site`) is refused like a cross-site one. Every spelling of a loopback bind address is recognised as loopback.
- **Validated tool calls (1.2.6).** A Tools-panel call goes through the same validation as ForgeGIS's MCP `tools/call`: an undeclared argument, a wrong type or an out-of-bounds value is refused with a named reason instead of silently ignored.
- **A separate door for the agent.** The browser session and the agent's bridge do not share a credential model, and the stronger one guards the ability to act on the map.

Built to Be Customized

Studio is a thin surface over stable engines, and that thinness is what makes it fast to tailor.

- **Capabilities flow through.** Operations arrive from the ForgeGIS catalog, stamped with kind and ports, and render as nodes automatically; an operation of a kind Studio already carries — raster, vector, or raster ↔ vector — is authorable, runnable and batchable without Studio code changes.
- **Workflows are data.** A pipeline tuned for a customer is a saved artifact that ships with a deployment and replays faithfully; curating a library of mission pipelines is configuration, not engineering.
- **The agent surface composes.** Extending what the agent can do on the map is adding a tool to the bridge, separate from both the engine and the browser.
- **Twins from the canvas.** A saved, parameterised raster pipeline built from twin-certified operations, with a default for every knob, can run as a digital twin with forcing series on its knobs, and be run, paused, forked and replayed from Studio's twin panel.

Deployment

Today. Studio is a Java server and a static frontend on one host. In the suite it is an opt-in component installed by the Seaglass Control Center, which lays down a private Temurin JRE 25 (the application targets Java 17), wires the agent bridge (shipped as a standalone executable, so no Node install), and starts Studio after the compute broker and the Globe and before ForgeMind, pointing it at the broker when that component is installed; it stops Studio gracefully, so a running batch records its state. **Open Studio** on the dashboard opens it in the browser on port 8081. The basemap and URL layers need a network; rasters render offline.

The evaluation build. Installed builds are evaluation builds under the licence accepted in the installer. Studio's top bar carries an evaluation chip — amber in the last two weeks, red once the period has ended — that gates nothing: Studio keeps running.

Accessibility (1.2.5). Modal focus is trapped and restored; the context menu is a real menu with arrow, Home and End keys; notifications are announced politely; focus rings are visible everywhere; every glyph-only button has a label and every disabled button says why.

Where deployment is headed. The single-host model is the first configuration, not the last. Since 1.2.6 Studio's raster batches already go to the suite's compute broker, which spreads them across GPU workers on one machine or a fleet. Serving several operators from one Studio, scheduling a shared VRAM budget across sessions, and the hosted security posture listed above remain a stated direction, with real work still ahead.

Known Limits in 1.2.6

- Single user, single host, and no login on the browser API — do not bind it off loopback.
- Branching and multi-input graphs run only in the Batch Runner. Only raster batches go to the compute broker; vector and mixed batches run locally, one at a time.
- Dry run does not check VRAM or parameters; a folder added through the picker must also be allowed for batch before it can be batched.
- Large Tools-panel geometry from a projected source is refused; vector tiles are refused for layers not in EPSG:4326 or 3857.
- Chat needs ForgeMind with the Studio surface wired; an agent key follows the most recent tab.
- The agent's dataset listing is capped and says when it is truncated, but cursor paging is not there yet.
- A rotated raster the reader cannot place is refused, not resampled — warp it to an axis-aligned grid first.

Specifications at a Glance

Role	The operator surface for the Forge suite — not a general-purpose GIS desktop
Frontend	React single-page app · MapLibre GL map · ReactFlow node editor
Server	Javalin (Jetty) HTTP and WebSocket, Java 17; the suite runs it on Temurin JRE 25
Bind	127.0.0.1:8081 by default; warns on every boot if rebound off loopback
Pipelines	Raster, vector, mixed; linear chains on the canvas, branching graphs in batch; one versioned format
Batch	Folder inputs; raster batches on the suite's compute broker when installed, the local runner otherwise; per-tile VRAM pre-flight (local), dry run, batch priority, durable report with termination reason
Data in	ForgeData catalog (embedded) · KML · KMZ · zipped Shapefile · XYZ, WMS, WMTS by URL · raster drops
Agent	ForgeMind chat on Studio's own map via the authenticated `studio-mcp` bridge (contract 1.0.0, 33 tools); scene context on every turn
Contracts	Pins the ForgeGIS and ForgeData MCP contracts it links (ForgeData 2.0.0); checked at boot
Persistence	Durable scene entities in the ForgeData catalog; pipeline library; batch history
Security	Loopback bind, sandboxed paths, upload and pixel caps, VRAM gate, bounded pool, per-operation timeout, hardened sessions, DNS-rebinding guard, separate agent credential
License	Proprietary. Part of the commercially licensed Forge suite, not sold separately; evaluation builds available

Licensing and Contact

ForgeGIS Studio is proprietary software of Seaglass Foundry LLC, distributed as an opt-in component of the Forge suite through the Seaglass Control Center and not sold separately. The Forge suite is commercially licensed; terms are provided on request, and evaluation builds are available to qualified teams. Nothing in this brief is a grant of rights.

Contact. rich@seaglassfoundry.com · seaglassfoundry.com — Seaglass Foundry™ reads every email.

ForgeGIS Studio™, ForgeGIS™, ForgeData™, ForgeMind™, Seaglass Globe™, Seaglass Control Center™, and Seaglass Foundry™ are trademarks of Seaglass Foundry LLC. React, MapLibre, ReactFlow, Javalin, Jetty, Java, Temurin, QGIS, and the Model Context Protocol are marks of their respective owners; reference to them does not imply endorsement. © 2026 Seaglass Foundry LLC. All rights reserved.